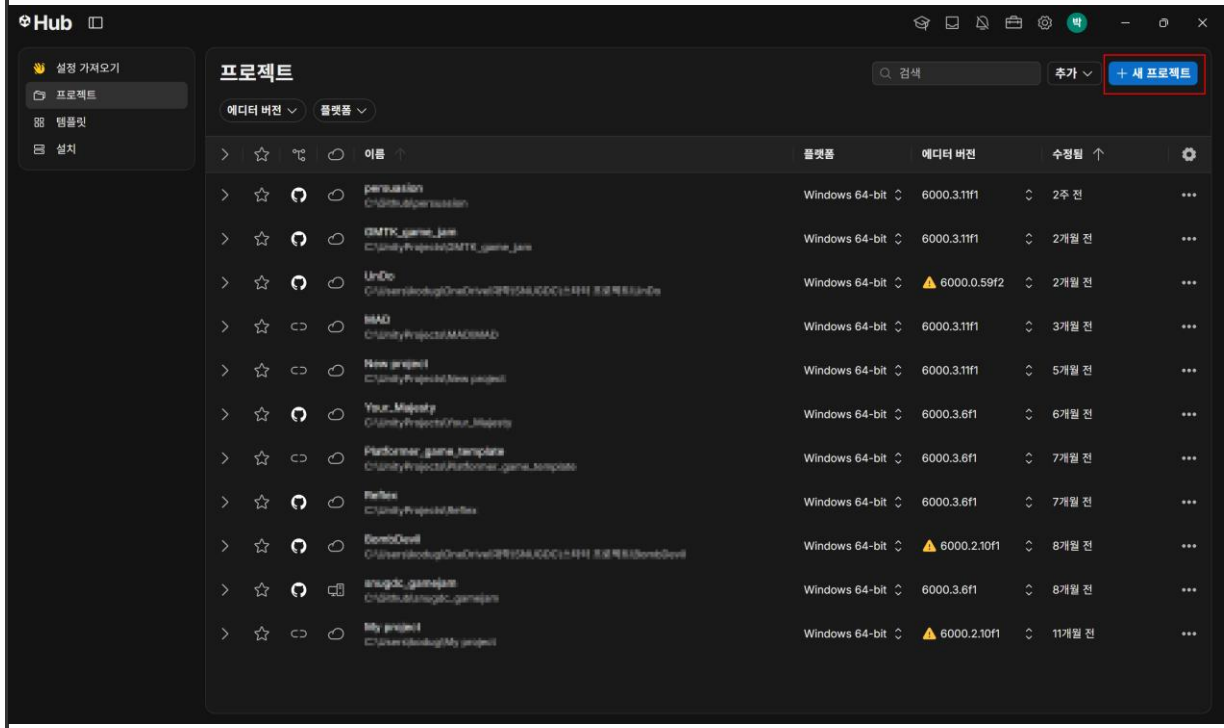
기초 Unity 스터디 2강

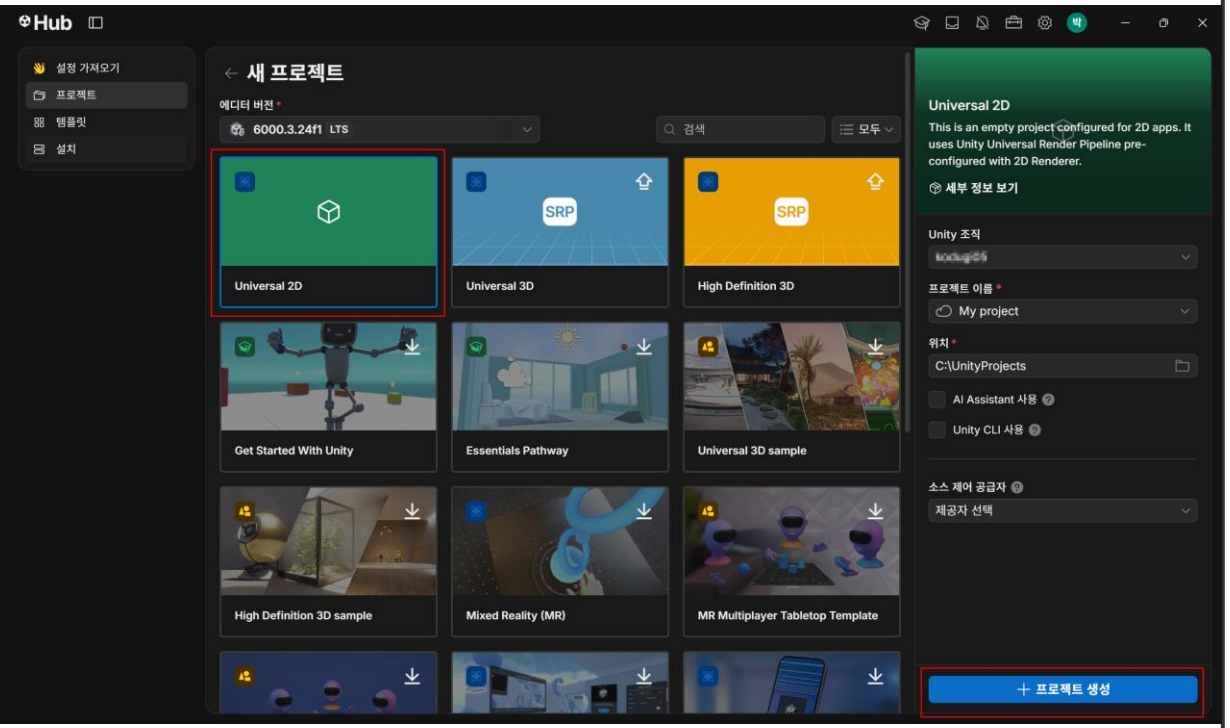
Unity 에디터 다뤄보기

1. 프로젝트 생성 및 인트로

프로젝트 생성

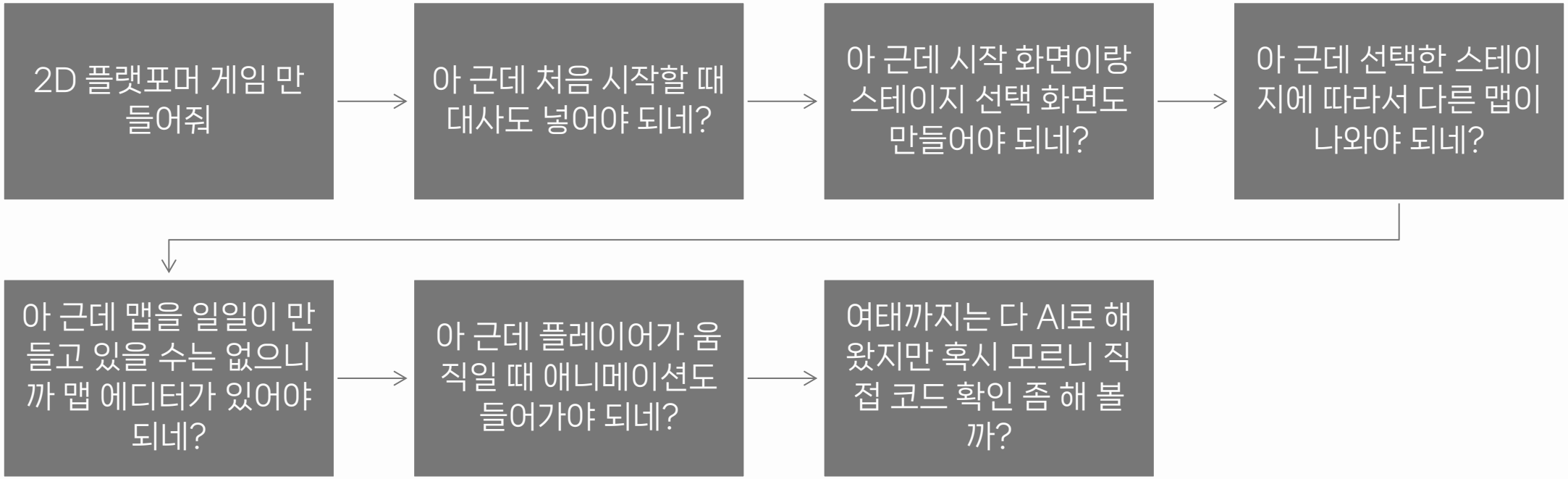


프로젝트-새 프로젝트

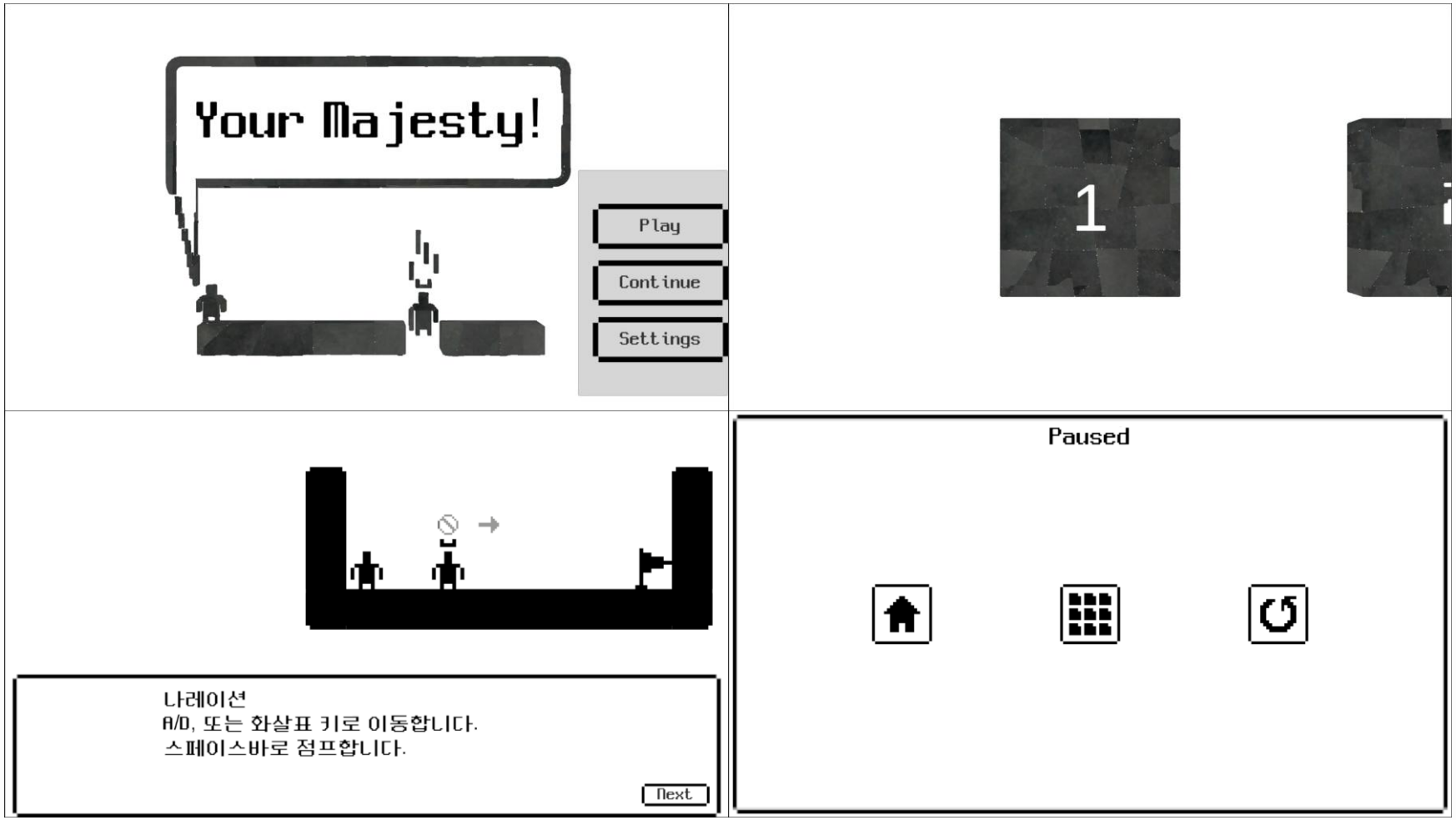


Universal 2D 선택
프로젝트 생성

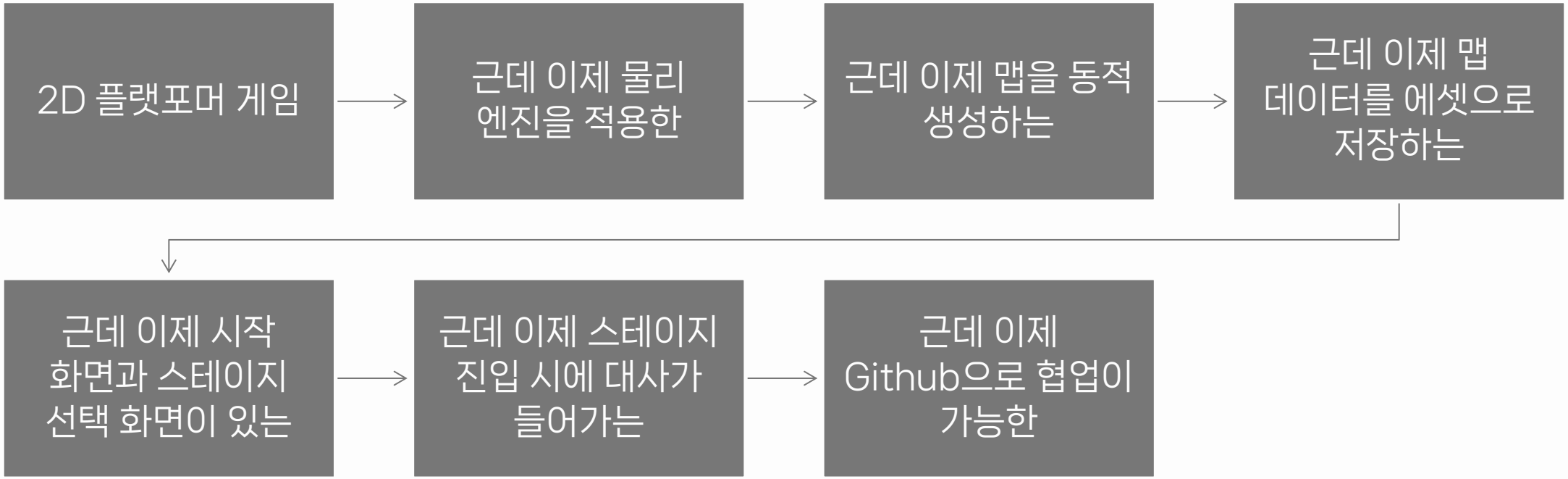
여기서 잠깐...


2D 플랫포머 게임 만들어줘
아 근데 처음 시작할 때 대사도 넣어야 되네?
아 근데 시작 화면이랑 스테이지 선택 화면도 만들어야 되네?
아 근데 선택한 스테이지에 따라서 다른 맵이 나와야 되네?
아 근데 맵을 일일이 만들고 있을 수는 없으니까 맵 에디터가 있어야 되네?
아 근데 플레이어가 움직일 때 애니메이션도 들어가야 되네?
여태까지는 다 AI로 해왔지만 혹시 모르니 직접 코드 확인 좀 해 볼까?

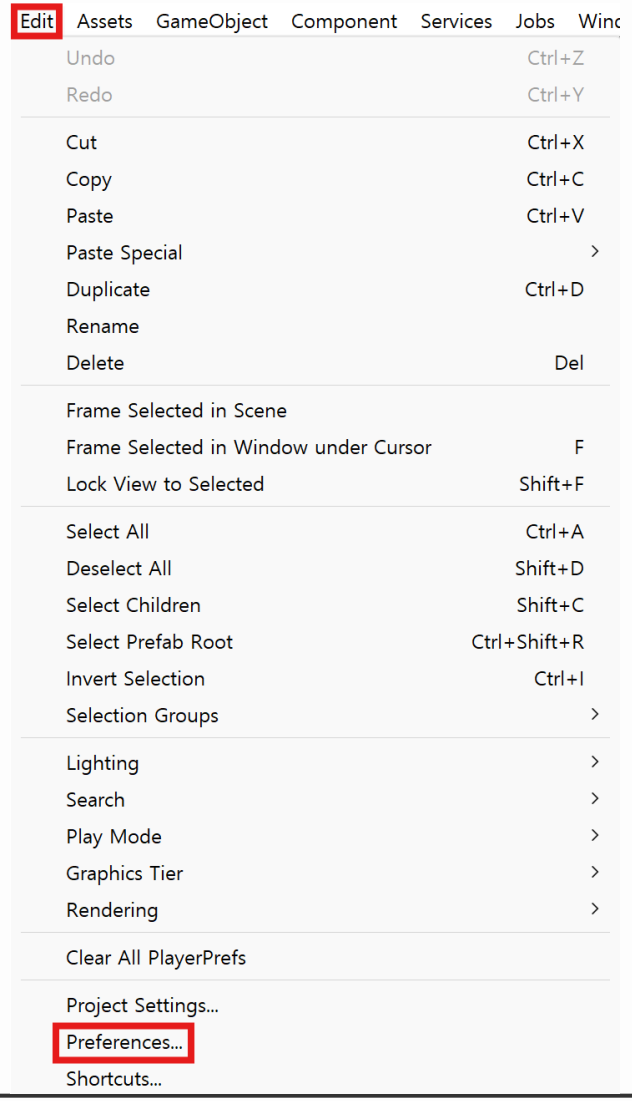
사실 경험담이었습니다


Your Majesty!
Play
Continue
Settings
1
나레이션
A/D, 또는 화살표 키로 이동합니다.
스페이스바로 점프합니다.
Next
Paused

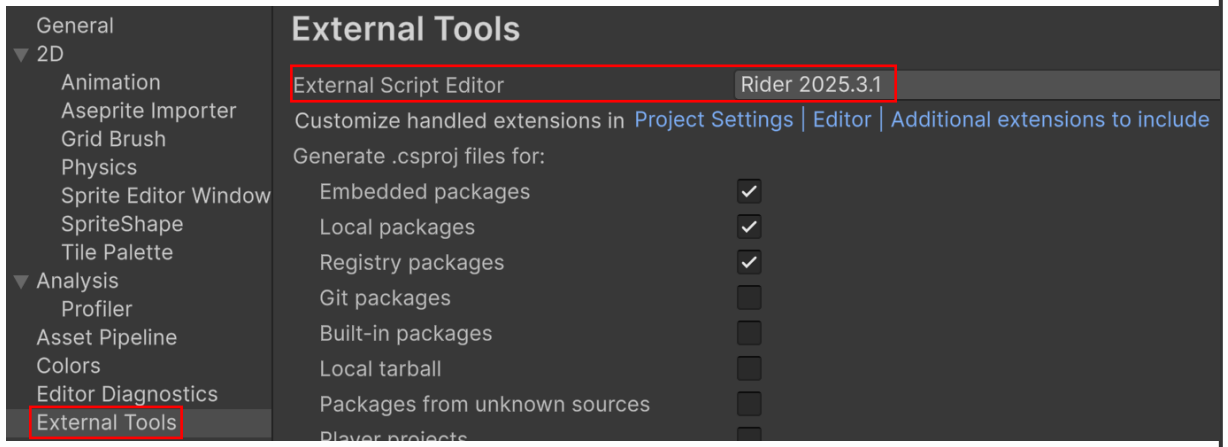
우리가 만들어볼 것


2D 플랫포머 게임
근데 이제 물리 엔진을 적용한
근데 이제 맵을 동적 생성하는
근데 이제 맵 데이터를 에셋으로 저장하는
근데 이제 시작 화면과 스테이지 선택 화면이 있는
근데 이제 스테이지 진입 시에 대사가 들어가는
근데 이제 Github으로 협업이 가능한

Unity와 Rider 연동하기

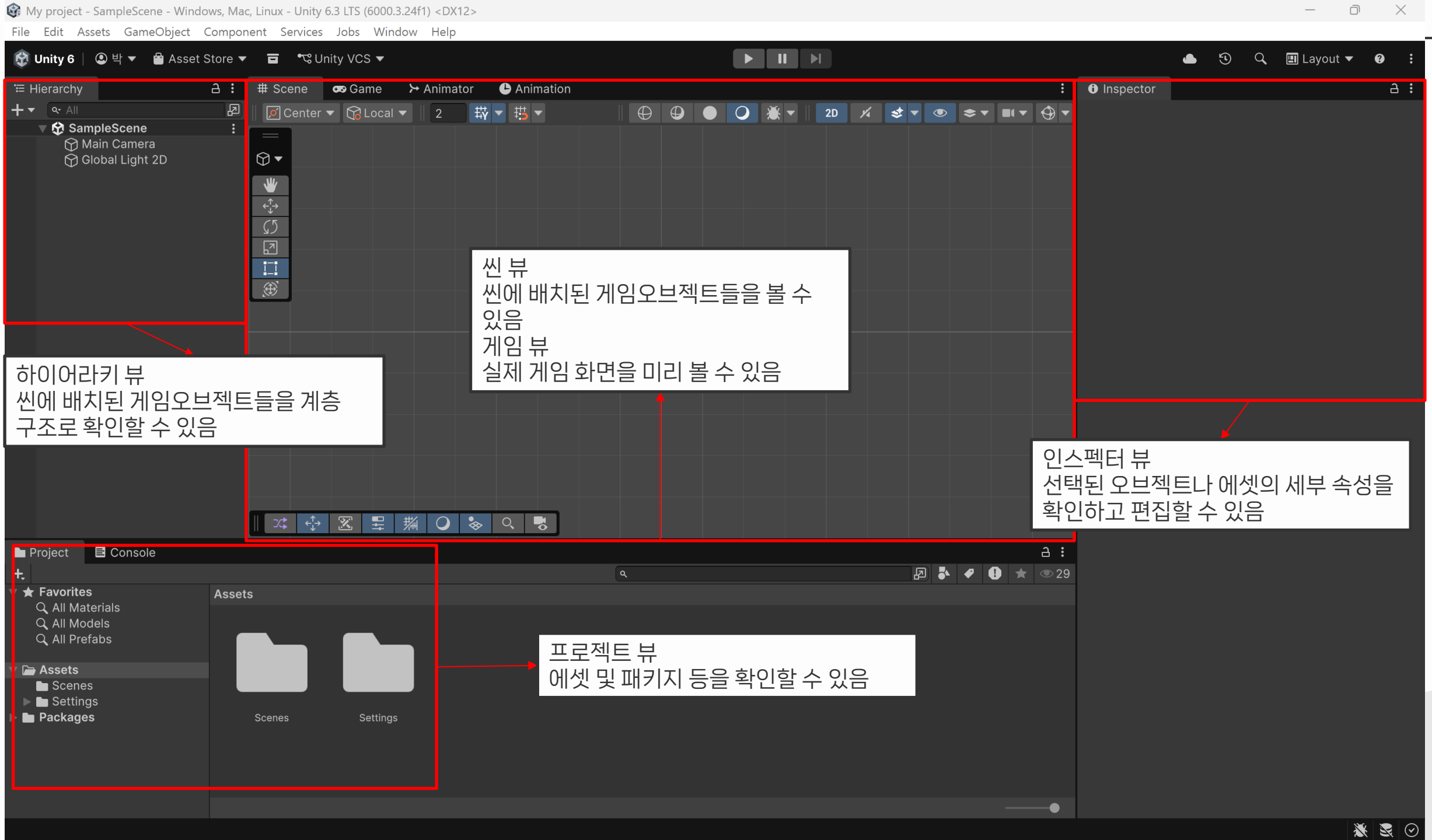


Edit-Preferences



External Tools-External Script Editor-Rider 선택

2. 에디터 둘러보기


My project - SampleScene - Windows, Mac, Linux - Unity 6.3 LTS (6000.3.24f1) <DX12>
File Edit Assets GameObject Component Services Jobs Window Help
Unity 6
Asset Store
Unity VCS
Layout
Hierarchy
All
SampleScene
Main Camera
Global Light 2D
Scene
Game
Animator
Animation
Center
Local
2D
Inspector
씬 뷰
씬에 배치된 게임오브젝트들을 볼 수 있음
게임 뷰
실제 게임 화면을 미리 볼 수 있음
하이어라키 뷰
씬에 배치된 게임오브젝트들을 계층 구조로 확인할 수 있음
인스펙터 뷰
선택된 오브젝트나 에셋의 세부 속성을 확인하고 편집할 수 있음
Project
Console
Favorites
All Materials
All Models
All Prefabs
Assets
Scenes
Settings
Packages
Assets
Scenes
Settings
프로젝트 뷰
에셋 및 패키지 등을 확인할 수 있음

씬과 게임오브젝트

- 씬(Scene): 게임의 서로 다른 장면
  - 서로 다른 기능을 수행하는 장면들은 주로 별개의 씬으로 구성
- 게임오브젝트(GameObject): 씬을 구성하는 기본 단위
  - 상호작용 가능한 모든 대상은 게임오브젝트
  - 배경 이미지, UI 등도 게임오브젝트의 일종
  - 눈에 보이지 않더라도 게임의 흐름을 조절하는 등의 역할을 담당하는 게임오브젝트도 있음
  - 씬에 배치된 게임오브젝트들은 씬이 시작될 때 생성되고, 다른 씬으로 전환되면 파괴됨

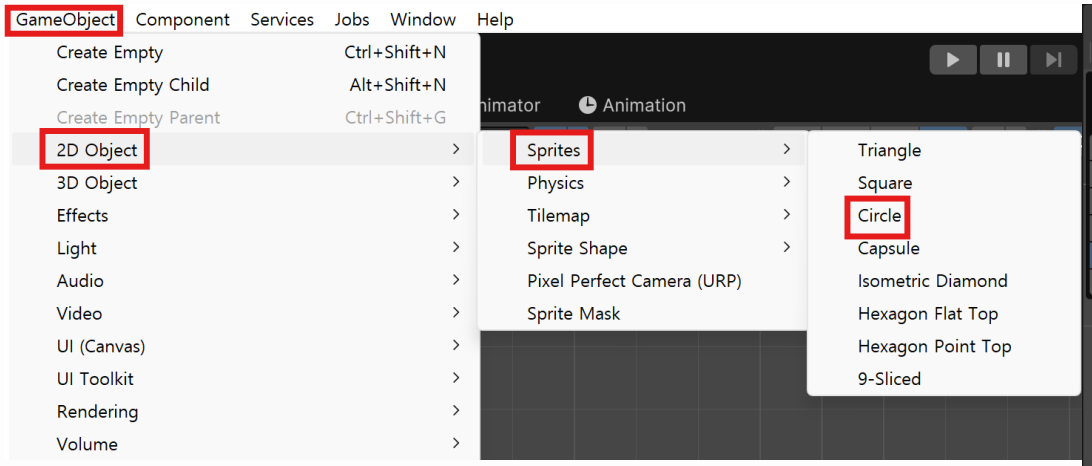
컴포넌트

- 컴포넌트(Component): 게임오브젝트에 붙어서 특정 효과를 내는 구성요소
- 컴포넌트의 예시
  - Transform: 오브젝트의 위치, 크기, 회전 등을 저장함, 모든 오브젝트가 필수로 가지는 컴포넌트
  - Sprite Renderer: 오브젝트의 이미지를 화면에 보이게 함
  - Rigidbody: 오브젝트가 물리 연산의 영향을 받게 함
  - Collider: 오브젝트에 충돌 판정을 추가
  - Audio Source: 오브젝트가 재생할 수 있는 음원을 추가

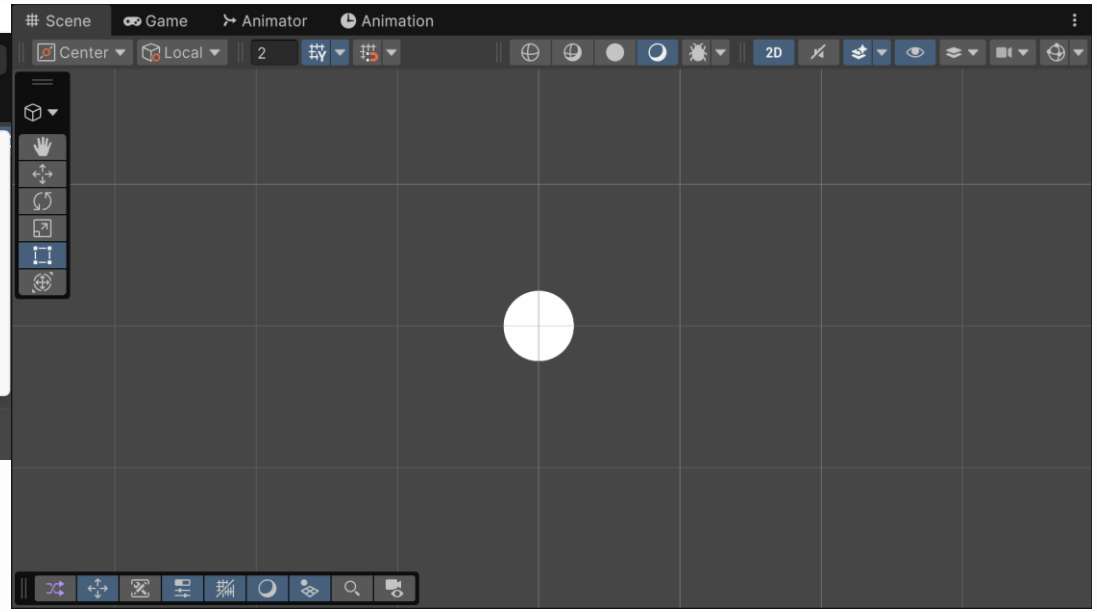
에셋

- 에셋(Asset): 게임에 사용할 수 있는 리소스
  - 이미지, 사운드, 폰트, 애니메이션 등
  - 에셋을 추가하는 것만으로는 게임 내에서 달라지는 것이 없음 → 에셋을 게임오브젝트에 적용해야 오브젝트가 달라지는 것을 볼 수 있음
  - ex) 이미지를 게임오브젝트에 적용하려면 Sprite Renderer의 이미지를 바꿔야 함

게임오브젝트 생성하기



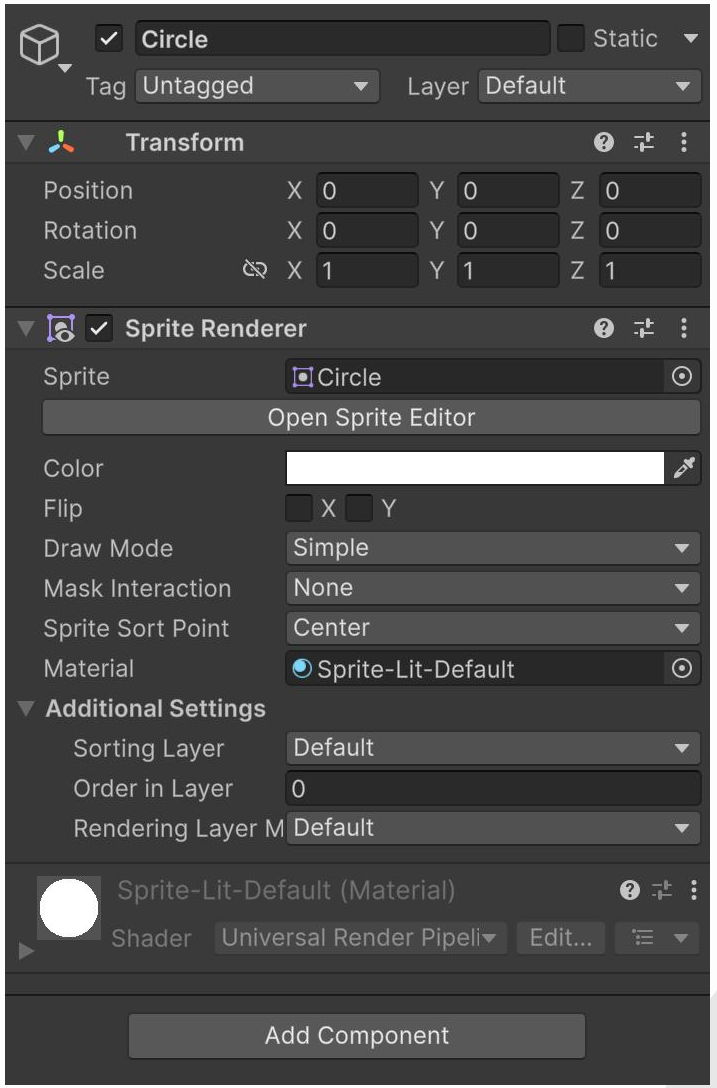
GameObject-2D Object-Sprites-Circle



원 모양의 게임오브젝트가 추가됨

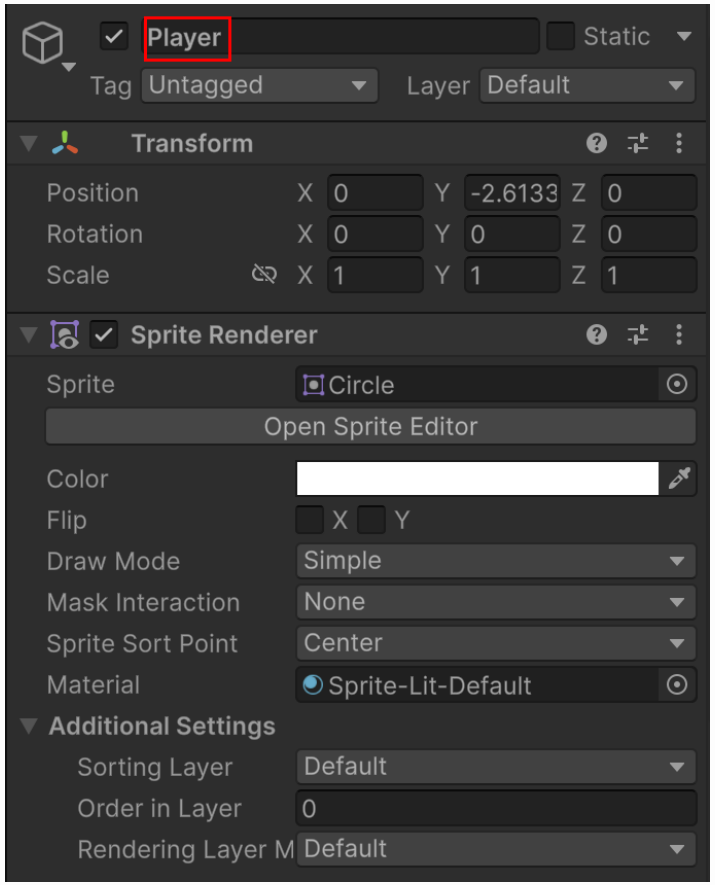
게임오브젝트 생성하기

- Circle 게임오브젝트
  - Transform, Sprite Renderer 컴포넌트를 가짐
- Transform에서 position을 바꾸면 Circle의 위치가 바뀌고, Scale을 바꾸면 Circle의 크기가 달라지는 것을 볼 수 있음(Rotation도 티가 나지는 않지만 각도가 달라짐)
- Sprite Renderer에서 Color를 바꾸면 Circle의 색이 바뀌고, Sprite를 바꾸면 오브젝트의 모습이 바뀜



게임오브젝트 생성하기

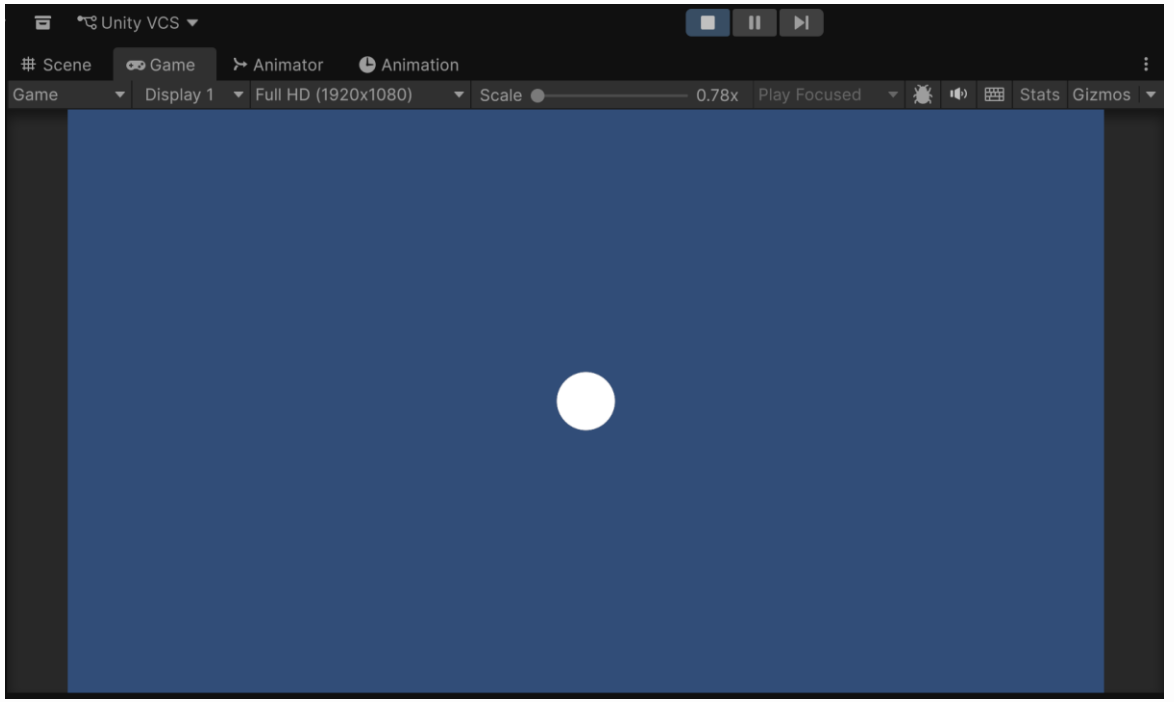
- 플레이어로 사용할 오브젝트이니, 이름을 Player로 변경



3. 스크립트로 오브젝트 움직이기

실행시켜 보기

- 시작 버튼을 누르면?
  - 아무 일도 일어나지 않음
  - 아무 것도 시키지 않았으니 당연한 결과
- 무슨 일이 일어나게 만들려면 어떻게 해야 할까?

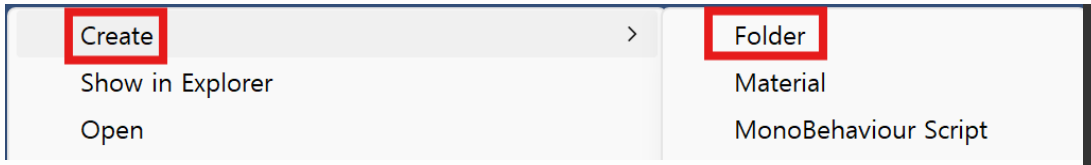


스크립트

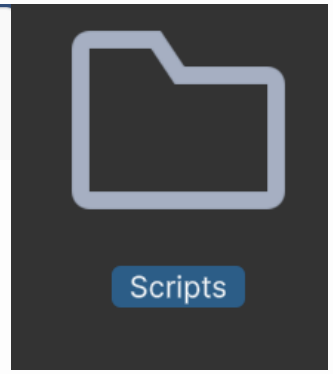
- 우리가 1주차에서 C# 문법을 공부한 이유!
- 스크립트도 에셋의 일종

- 스크립트를 오브젝트에 적용하면 컴포넌트가 됨
  - 스크립트를 통해 오브젝트의 다른 컴포넌트의 값을 읽거나 변경할 수 있고, 다른 오브젝트들에 영향을 미칠 수도 있음
  - '방향키를 누르면 방향에 따라 움직이게 하는 컴포넌트' 같은 걸 일일이 Unity에서 지원해줄 수 없으니 우리가 직접 만든다고 생각하면 됨

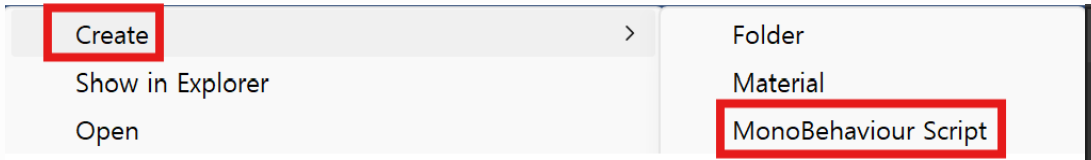
스크립트 생성하기



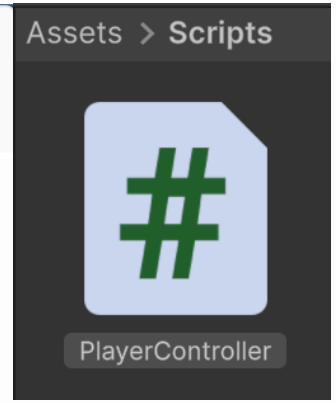
에셋 뷰 우클릭-Create-Folder



Scripts 폴더 생성

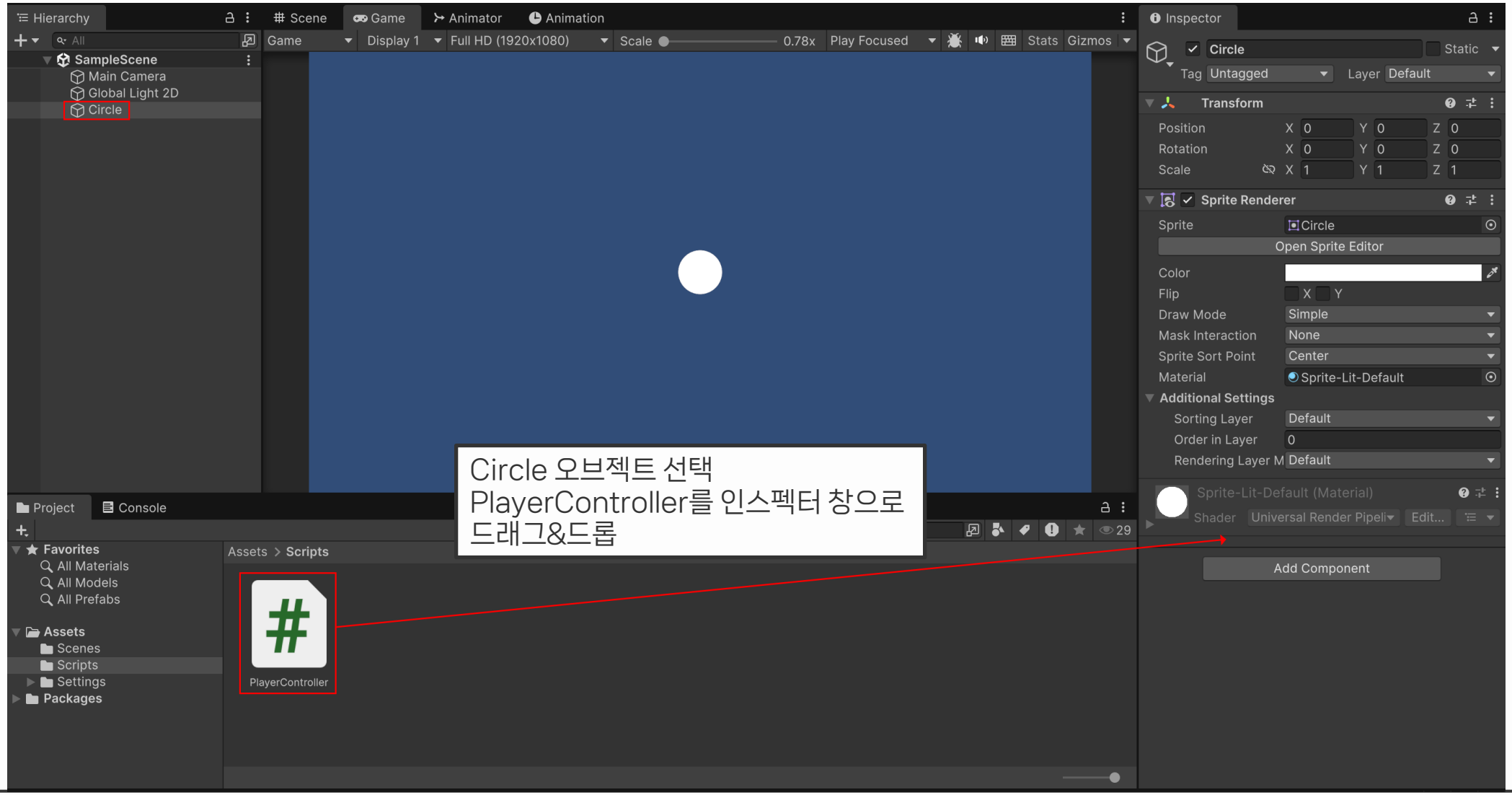


Scripts 폴더 안에서 우클릭-Create-MonoBehavior Script



PlayerController 스크립트 생성

스크립트 적용하기


Hierarchy
Scene
Game
Animator
Animation
Inspector
SampleScene
Main Camera
Global Light 2D
Circle
Game
Display 1
Full HD (1920x1080)
Scale
0.78x
Play Focused
Stats
Gizmos
Circle
Static
Tag
Untagged
Layer
Default
Transform
Position
Rotation
Scale
Sprite Renderer
Sprite
Circle
Open Sprite Editor
Color
Flip
Draw Mode
Simple
Mask Interaction
None
Sprite Sort Point
Center
Material
Sprite-Lit-Default
Additional Settings
Sorting Layer
Default
Order in Layer
Rendering Layer M
Sprite-Lit-Default (Material)
Shader
Universal Render Pipeli
Edit...
Add Component
Project
Console
Favorites
All Materials
All Models
All Prefabs
Assets
Scenes
Scripts
Settings
Packages
Assets > Scripts
PlayerController
Circle 오브젝트 선택
PlayerController를 인스펙터 창으로
드래그&드롭

스크립트 구조 살펴보기

- PlayerController.cs를 Rider로 열면 보이는 모습

C# PlayerController.cs

```
1   using UnityEngine;
2
3   public class PlayerController : MonoBehaviour
4   {
5       // Start is called once before the first execution of Update after the MonoBehaviour is created
6       void Start()
7       {
8
9       }
10
11      // Update is called once per frame
12      void Update()
13      {
14
15      }
16  }
17
```

UnityEngine 네임스페이스 사용
MonoBehavior를 사용하기 위해 필요

PlayerController 클래스
: MonoBehavior:
MonoBehavior라는 클래스를 상속한다는 의미
MonoBehavior에 정의된 Start, Update 등의 함수를 쓸 수 있게 만듦

Start 함수
Unity 엔진에서 내부적으로, 오브젝트가 생성될 때 한 번 실행

Update 함수
Unity 엔진에서 내부적으로, 매 프레임마다 반복적으로 실행

스크립트로 오브젝트 움직이기

- 우리가 해볼 것: 방향키를 누르면 오브젝트가 움직이게 만들기
- '방향키를 누르는 것을 감지'하는 기능과, '오브젝트를 움직이는' 기능이 필요함

- 방향키 누름 감지는 Input.GetKey를 통해, 오브젝트 이동은 transform.Translate를 통해 할 수 있음

방향키 입력 감지

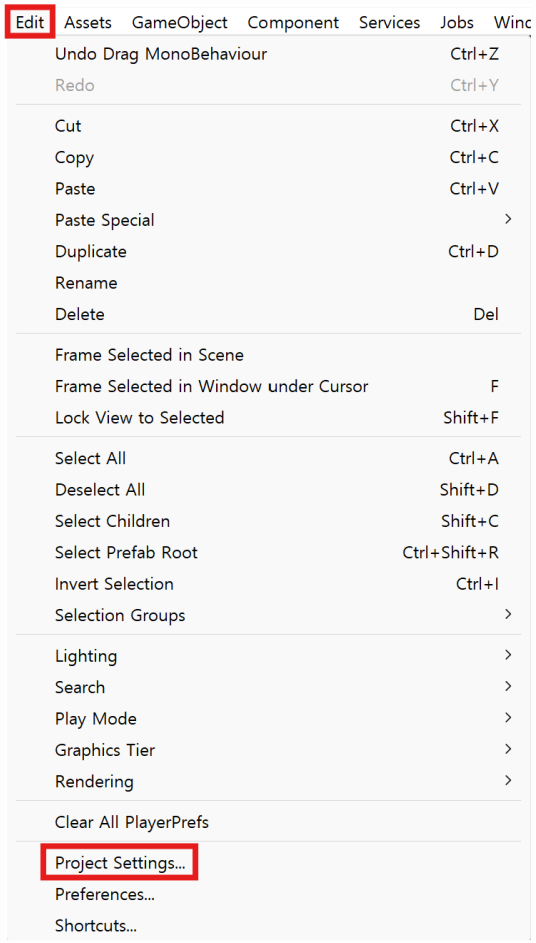
- 키보드나 마우스 등의 입력은 Input 클래스에서 처리함

- **Input.GetKeyDown(Keycode key)**: key에 해당하는 키가 눌린 프레임에 한 번만 true
- **Input.GetKey(Keycode key)**: key에 해당하는 키가 눌려 있는 프레임마다 true
- **Input.GetKeyUp(Keycode key)**: key에 해당하는 키를 눌렀다가 뗀 프레임에 한 번만 true

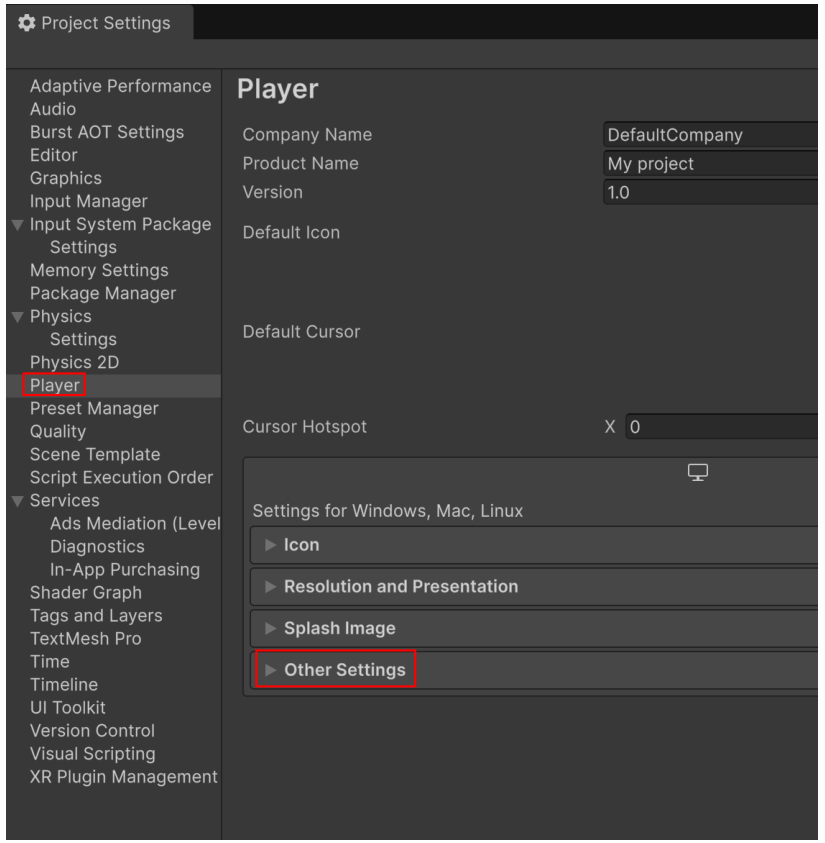
Input Manager를 쓰기 전 사전 준비

- Unity에서 입력을 받는 방법이 크게 두 가지 방법으로 나뉨
  - Input Manager: 기존 입력 방식, 단순함
  - Input System: 새로운 입력 방식, 복잡하지만 서로 다른 입력 방식을 사용하는 기기에서도 같은 방식으로 입력을 받을 수 있게 함
- 일단은 단순성을 위해 Input Manager를 쓸 예정
- 다만 Unity의 기본 프로젝트 세팅은 Input System을 사용하는 것으로 되어 있어서, 이걸 변경해야 Input Manager로 입력을 받을 수 있음

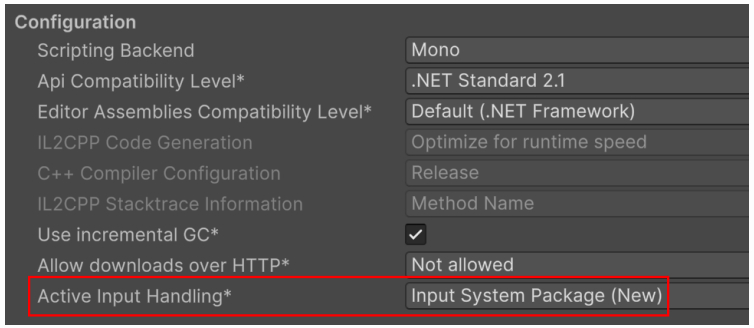
Input Manager를 쓰기 전 사전 준비



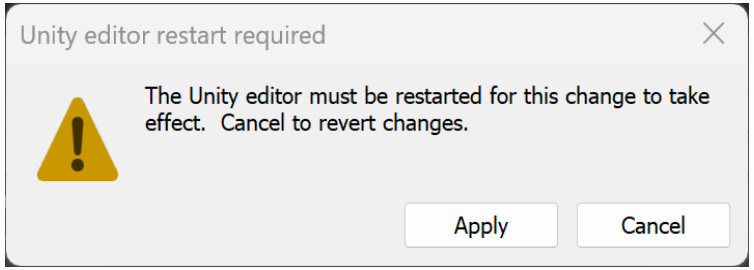
Edit-Project Settings



Player-Other Settings



Configuration-Active Input Handling에서 Input System Package를 Both로 변경



Apply를 누르면 에디터가 재시작됨

플레이어 이동

- 게임오브젝트의 좌표는 Transform 컴포넌트를 통해 접근 가능

- 게임오브젝트의 컴포넌트에 접근하는 방법: **(오브젝트 이름).GetComponent<(컴포넌트 타입)>()**
  - ex) **player.GetComponent<Transform>();**
  - 자기 자신의 컴포넌트에 접근할 때는 오브젝트 이름 생략 가능
- 단, Transform 컴포넌트는 모든 게임오브젝트가 가지고 있는 컴포넌트이므로 **GetComponent** 없이 **transform**으로만 써도 접근 가능하게 되어 있음

- **transform.Translate(Vector3 translation):** transform의 좌표를 translation만큼 평행이동시킴

스크립트로 오브젝트 움직이기

```
using UnityEngine;

애셋 사용 위치가 없습니다
public class PlayerController : MonoBehaviour
{
    // Start is called once before the first execution of Update after the MonoBehaviour is created
    이벤트 함수
    void Start()
    {

    }

    private float _speed = 5.0f;

    // Update is called once per frame
    이벤트 함수
    void Update()
    {
        if (Input.GetKey(KeyCode.LeftArrow))
        {
            transform.Translate( translation: Vector3.left * _speed);
        }

        if (Input.GetKey(KeyCode.RightArrow))
        {
            transform.Translate( translation: Vector3.right * _speed);
        }
    }
}
```

- 이 코드를 직접 타이핑해서 실행해보고, 문제가 뭔지 파악해보자.

스크립트로 오브젝트 움직이기

- 코드 분석

```
private float _speed = 5.0f;
```

속도 값을 변수로 설정

```
void Update()
```

입력을 감지하고 오브젝트를 움직이는 코드는 매 프레임 실행해야 하므로 Update 안에 들어감

```
Input.GetKey(KeyCode.LeftArrow)
```

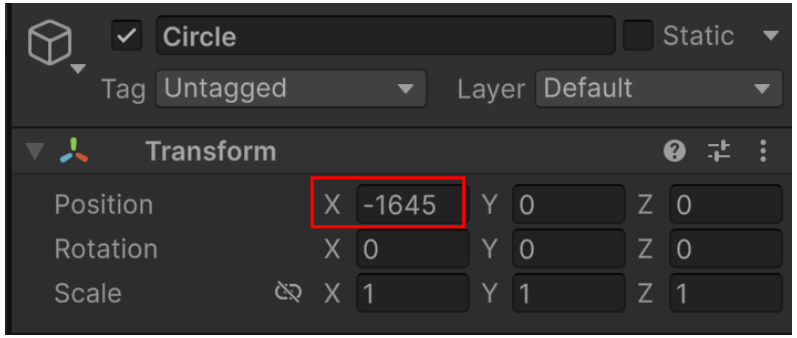
왼쪽 화살표 키를 누르면

```
Vector3.left * _speed
```

Vector3: Unity에서 Transform 등에 쓰이는 3차원 벡터
Vector3.left: (-1, 0, 0)
Vector3와 스칼라 값의 곱은 Vector3를 스칼라 배 하는 효과

문제가 뭘까?

- 표면상으로 보이는 문제: 방향키를 누르면 오브젝트가 잠깐 보였다가 사라짐
- Circle의 좌표를 확인해보자



- 일반적으로 오브젝트가 화면에 보이는 범위는 X좌표 -8~8, Y좌표 -5~5 정도임
- 그 범위를 한참 벗어났으니 안 보이는 것
- 처음에는 잠깐 보였다가 사라진다는 것은 오브젝트의 이동 속도가 너무 빠르다는 뜻

- 겉으로 보이는 문제와 실제 문제의 원인은 크게 다를 수 있다!

문제가 뭘까?

- 이동 속도가 왜 빠를까?
- Update는 '매 프레임마다' 실행된다고 했음
- 보통 일반적인 게임의 FPS는 30~60 정도인데, 이는 초당 30~60 프레임이 실행된다는 뜻
- 지금 Circle은 매 프레임마다 +-(5, 0, 0)만큼을 이동하는데, 이는 초당 150~300만큼을 이동한다는 뜻임
- 속도를 낮추면 해결되는 문제라고 생각할 수 있으나, 더 큰 문제가 있음
- 게임의 FPS는 일정하지 않으므로, 지금처럼 프레임 당 일정 거리를 움직이는 방식은 FPS에 따라 같은 시간 동안이더라도 이동한 거리가 달라짐

문제 해결

- 문제 해결 방향: '매 프레임 당' 같은 변위만큼 이동하는 것이 아니라, '매 시간 당' 같은 변위만큼 이동시킨다
- 프레임 간의 시간 간격이 좁으면 적게 이동하고, 넓으면 크게 이동하면 됨
- 프레임 간의 시간 간격: Time.deltaTime으로 알 수 있음

- ⇒ 해결 방안: 이동 방향을 나타내는 벡터에 Time.deltaTime을 곱한다

```
void Update()
{
    if (Input.GetKey(KeyCode.LeftArrow))
    {
        transform.Translate( translation: Vector3.left * _speed * Time.deltaTime);
    }

    if (Input.GetKey(KeyCode.RightArrow))
    {
        transform.Translate( translation: Vector3.right * _speed * Time.deltaTime);
    }
}
```

실습 1: 이것저것 바꿔 보기

- 1. 속도를 더 느리게 해 보자
- 2. 위, 아래 이동도 추가해보자

- (문제 1, 2 내용은 다음 내용과는 이어지지 않으니, 해 본 뒤에는 코드를 원상복구하자)

실습 1: 이것저것 바꿔 보기

- 문제 1 정답

```
private float _speed = 2.0f;
```

- 문제 2 정답

```
if (Input.GetKey(KeyCode.UpArrow))
{
    transform.Translate( translation: Vector3.up * _speed * Time.deltaTime);
}

if (Input.GetKey(KeyCode.DownArrow))
{
    transform.Translate( translation: Vector3.down * _speed * Time.deltaTime);
}
```

아직 끝이 아니다

- 잘 되기는 하지만, 뭔가 아쉬운 점이 있음
  - 방향을 바꾸거나 정지했다가 움직일 때, 이동이 부드럽지 않음
  - (어쩌다 보니 여기서는 큰 문제가 없었지만) 여러 방향키를 동시에 눌렀을 경우에 대한 처리가 복잡함
  - ex) 왼쪽 키를 먼저 누르고 있다가 다음에 오른쪽 키를 같이 누르면 어떻게 처리해야 할까? → 정지하는 게 상식적이지만, 방향키 이동을 어떻게 구현하느냐에 따라 정지하지 않고 계속 왼쪽으로 움직이거나, 갑자기 오른쪽으로 이동할 수도 있음

아직 끝이 아니다

- 방향키 입력을 쉽게 받는 방법: Input.GetAxis

- float GetAxis(String axisName): axisName에 따라 축별 입력값(실수)을 반환
  - "Horizontal": 가로 방향(좌우) 입력, -1~1 사이의 연속적인 값
  - "Vertical": 세로 방향(상하) 입력, -1~1 사이의 연속적인 값

실습 2: Input.GetAxis로 코드 바꾸기

- 직접 앞서 작성했던 코드를 Input.GetAxis를 사용하도록 바꿔 보자

- 힌트
  - 조건문으로 방향을 판단할 필요가 없다(벡터에서는 부호가 곧 방향이다!)
  - 속도를 연속적으로 만들려면 속도가 GetAxis의 반환값에 '비례'해야 하지 않을까?
- 추가 힌트
  - (1, 0, 0) 벡터에 GetAxis 값을 곱하면 된다

실습 2: Input.GetAxis로 코드 바꾸기

- 정답

```
void Update()
{
    transform.Translate( translation: _speed * Time.deltaTime * Input.GetAxis("Horizontal") * Vector3.right);
}
```

4. 물리 엔진 적용하기

물리 상호작용 구현

- 방향키를 누르면 좌우로 움직이게 하는 건 성공했다
- 이제 해야 할 것
  - 스페이스바를 누르면 점프하게 만들기
  - 플레이어가 외력(중력, 수직항력 등)의 영향을 받게 하기

- 마찬가지로 transform.Translate를 쓰면 되지 않을까?
- 몇 가지 문제
  - 가속도를 일일이 계산해서 프레임마다 속도를 변화시켜야 한다
  - 플레이어가 바닥이나 벽에 닿으면 움직임이 달라져야 하는데, 어떻게 달라져야 하는지도 모를 뿐더러 닿았는지 여부는 더더욱 알기 어렵다

물리 상호작용 구현

- Unity에서는 복잡한 물리 연산을 대신해주는 컴포넌트들이 있음

- Rigidbody
  - '강체'라는 뜻
  - 강체: 변형이 일어나지 않는 물체 → 이동, 회전 등은 가능하지만 형태는 바뀌지 않음
  - Unity에서 Rigidbody는 오브젝트가 물리적 상호작용의 대상이 되도록 하는 컴포넌트

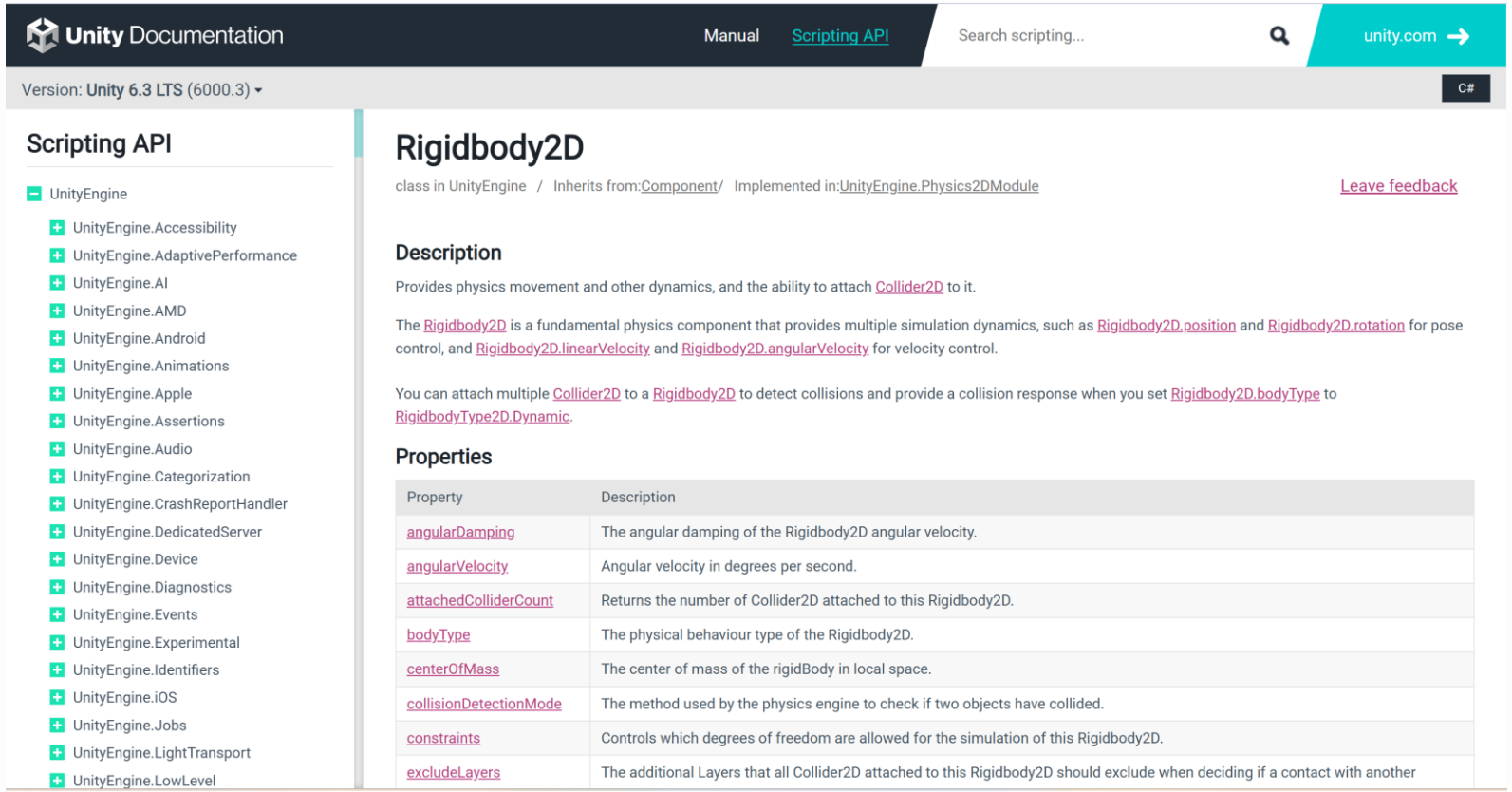
- Collider
  - 오브젝트에 충돌 판정이 작용하게 함

- 이 두 기능을 사용하면 스크립트 없이도 플레이어가 외력의 영향을 받게 할 수 있음

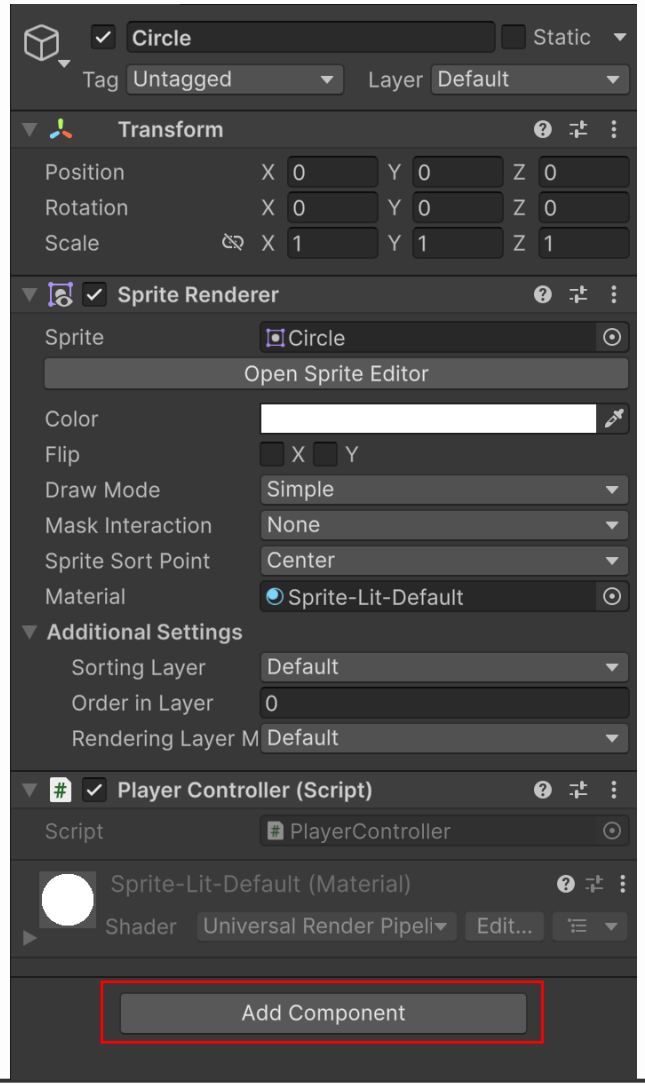
- 우리는 2D 게임을 만드는 중이므로, 2D 게임용 컴포넌트인 Rigidbody2D와 Collider2D를 사용함

물리 상호작용 구현

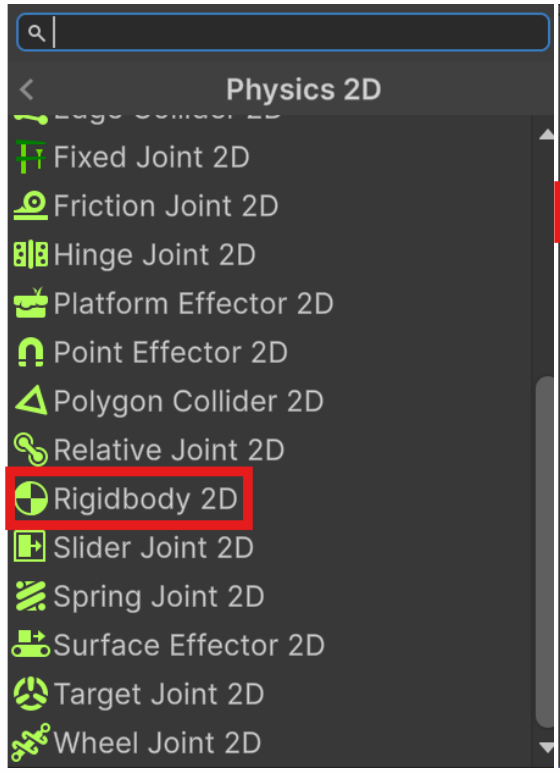
- 컴포넌트에 대한 자세한 정보 알아보기: Unity 공식 문서 참조
- Rigidbody2D
- Collider2D



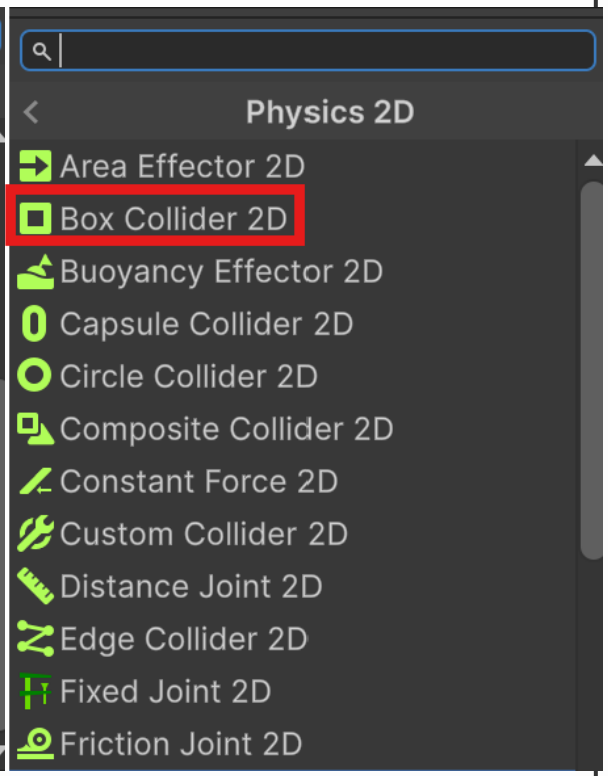
RigidBody2D와 Collider2D 추가하기



Circle 선택-Add Component



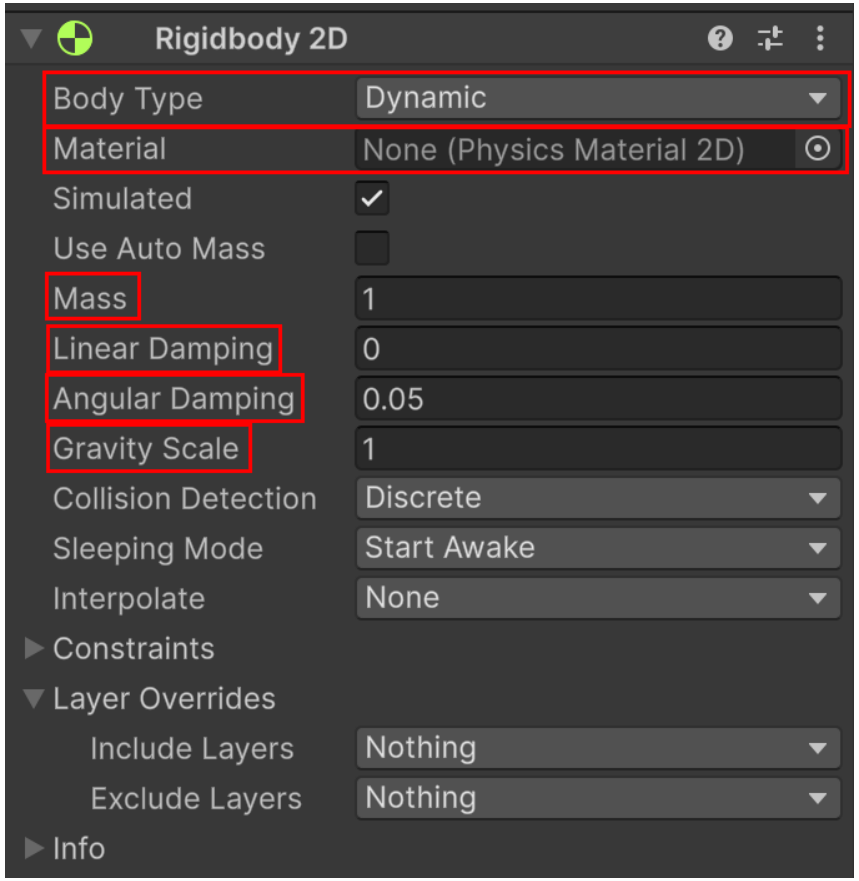
Physics 2D-RigidBody 2D 선택



같은 방법으로 Box Collider 2D도 추가

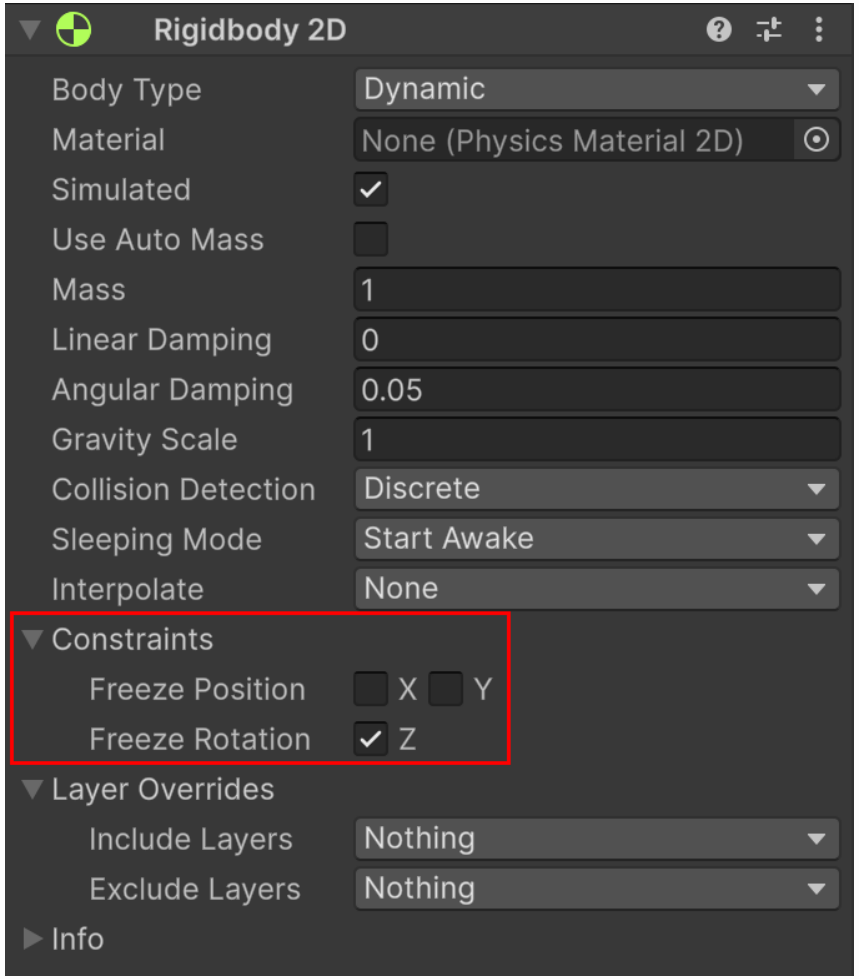
RigidBody2D 살펴보기

- Body Type
  - Dynamic: 외력에 의해 움직일 수 있음
  - Kinetic: 외력에 의해 움직이지 않음
- Mass: 질량
- Linear Damping: 이동 속도 감쇠
- Angular Damping: 회전 속도 감쇠
- Gravity Scale: 오브젝트에 적용되는 중력의 세기



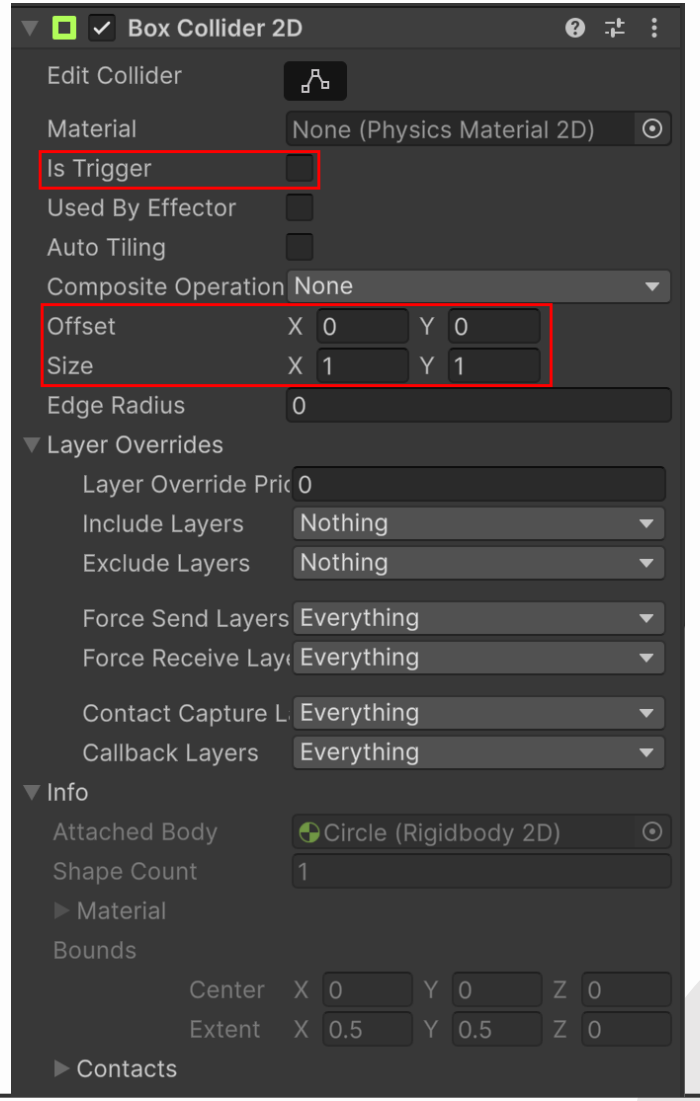
RigidBody2D 살펴보기

- Constraints
  - Freeze Position: 체크 시 각각 X축/Y축 움직임을 막음
  - Freeze Rotation: 체크 시 회전을 막음
- 플레이어는 상하좌우로 이동만 하고 회전하지는 않으므로 Freeze Rotation에 체크



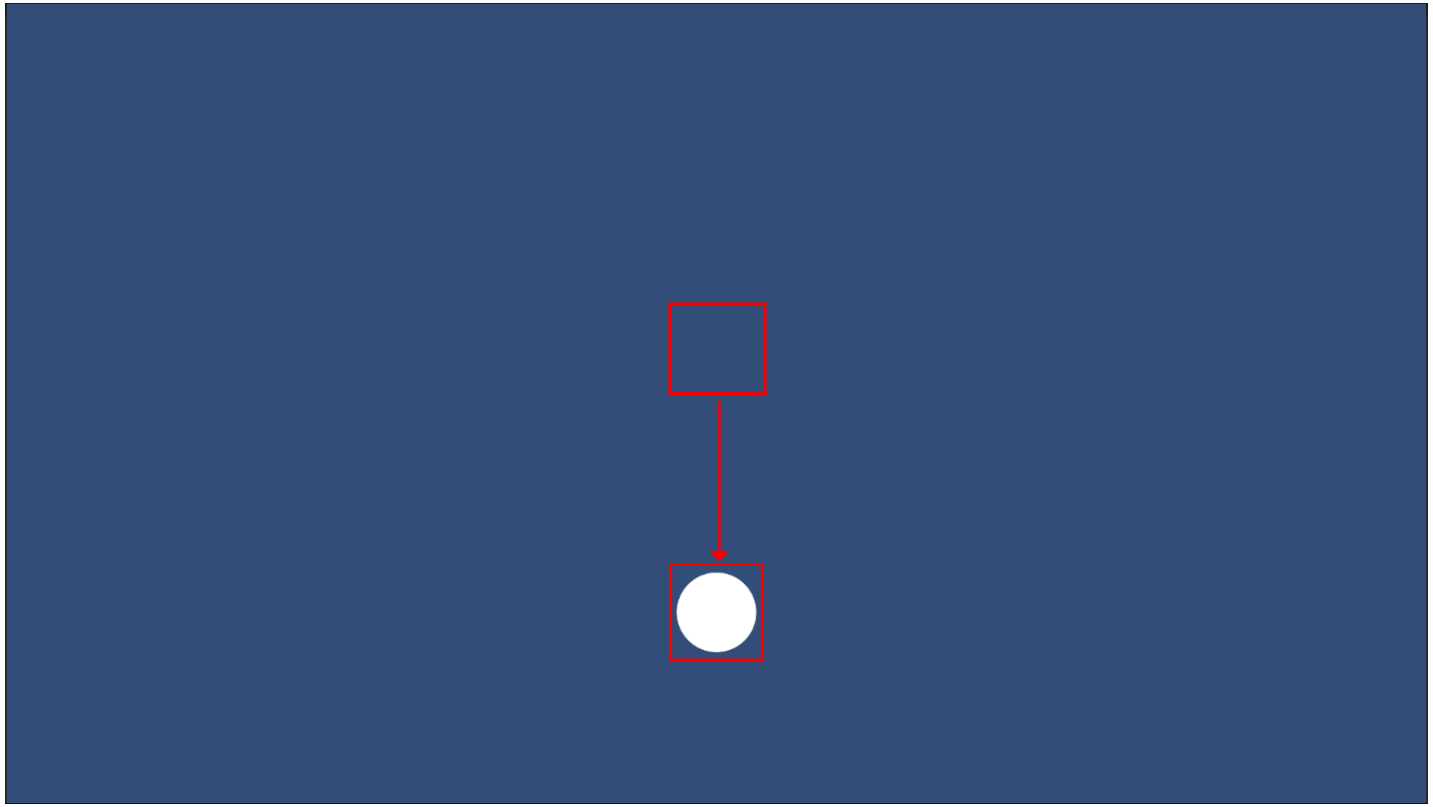
Box Collider 2D 살펴보기

- Is Trigger
  - 체크할 경우 다른 오브젝트들이 뚫고 지나갈 수 있으며, 이때 충돌 판정이 되지 않고 Trigger 판정이 됨
  - 체크하지 않을 경우 다른 오브젝트들이 뚫고 지나갈 수 없으며, 충돌 판정이 됨
- Offset과 Size
  - Offset은 중심으로부터 콜라이더의 상대적 위치를 나타냄
  - Size는 오브젝트에 대한 상대적 크기를 나타냄
- 콜라이더를 확인하는 방법: Edit Collider 클릭 후 씬 뷰 선택



물리 엔진 적용하기

- 실행해보면 실제로 Circle이 중력의 영향을 받아 떨어지는 것을 확인할 수 있음



실습 3: 발판 만들기

- 플레이어가 딛고 서 있을 수 있는 발판을 만들어 보자

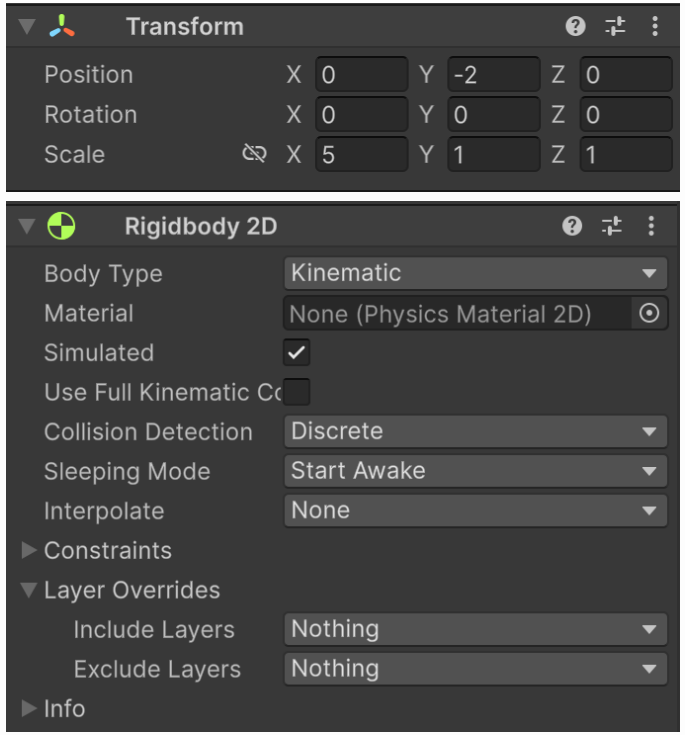
- 힌트
  - 발판 자체는 외력의 영향을 받으면 안 된다
  - 모양은 그림처럼 넓은 직사각형 모양
  - 위의 사실만 빼면 나머지는 플레이어와 똑같다
- 추가 힌트
  - 외력의 영향을 받지 않으려면 **Rigidbody2D**의 Body Type이...?
  - GameObject-2D Object-Sprites-Square
  - 크기는 **Transform**의 Scale에서 바꿀 수 있다



실습 3: 발판 만들기

- 정답
  - Transform 컴포넌트에서 위치는 플레이어 아래로, x축 scale은 1보다 크게 설정

  - RigidBody에서 Body Type은 Kinematic으로 설정
  - Box Collider 추가



점프 기능 만들기

- 이제 외력 작용과 충돌 판정이 모두 가능해졌으니, 점프 기능을 구현할 수 있음

- 점프는 '위쪽으로 외력을 작용시키는' 것으로 볼 수 있음
- Rigidbody2D 컴포넌트의 AddForce(Vector2 force): force만큼 힘을 가함

점프 기능 만들기

```
public class PlayerController : MonoBehaviour
{
    // Start is called once before the first execution of Update after the MonoBehaviour is created
    이벤트 함수
    void Start()
    {
        _rb = GetComponent<Rigidbody2D>();
    }

    private float _speed = 5.0f;
    private float _jumpForce = 500.0f;
    private Rigidbody2D _rb;

    // Update is called once per frame
    이벤트 함수
    void Update()
    {
        transform.Translate( translation: _speed * Time.deltaTime * Input.GetAxis("Horizontal") * Vector3.right);

        if (Input.GetKeyDown(KeyCode.Space))
        {
            _rb.AddForce(Vector2.up * _jumpForce);
        }
    }
}
```

코드 분석

```
void Start()
{
    _rb = GetComponent<Rigidbody2D>();
}
```

Start에서 GetComponent로 Rigidbody2D 컴포넌트를 _rb에 저장함
→ GetComponent는 비싼(시간이 오래 소요되는) 연산이기에 매 프레임 실행하기에는 부담되므로 맨 처음 한 번만 수행해서 컴포넌트를 저장해두는 것

```
Input.GetKeyDown(KeyCode.Space)
```

스페이스바가 눌린 그 프레임에서만 점프가 되고, 그 이후에는 점프가 되면 안 되므로 Input.GetKeyDown 사용

```
_rb.AddForce(Vector2.up * _jumpForce);
```

점프는 위쪽으로 힘을 가하는 것이므로 AddForce에 Vector3.Up * _jumpForce 값을 넣음

점프력을 조절하려면 _jumpForce를, 플레이어에 작용하는 중력가속도를 바꾸려면 Rigidbody2D의 Gravity Scale을 바꾸면 됨

땅과의 충돌 판정

- 문제점: 스페이스바를 계속 누르면 계속 점프가 됨
- 해결 방법: 한 번 점프를 하면 땅에 닿기 전까지는 다시 점프하지 못하게 만들자

- '땅에 닿는' 것을 판정하려면 Collider2D를 이용하면 됨

- Collider2D 관련 이벤트 함수
  - **OnCollisionEnter2D(Collision2D collision)**: 충돌이 시작된 프레임에 호출되는 함수
  - **OnCollisionExit2D(Collision2D collision)**: 충돌이 끝난 프레임에 호출되는 함수
- Collision2D
  - 충돌과 관련된 정보를 저장하는 클래스
  - **collider/otherCollider**: 충돌한 두 Collider2D(상대/본인)
  - **rigidbody/otherRigidbody**: 충돌한 두 Rigidbody2D(상대/본인)
  - **gameObject**: 충돌한 상대 게임오브젝트

땅과의 충돌 판정

- 원래는 충돌한 게임오브젝트가 땅인지 아닌지도 판정해야 하지만, 이 부분은 다음 주차에서 다룰 예정
- 현재는 플레이어가 충돌할 수 있는 다른 오브젝트가 땅 뿐이므로 충돌하면 무조건 땅이라고 간주

- 현재 공중에 떠 있는지 여부를 저장하는 변수를 두어서, 공중에 떠 있지 않을 때만 점프가 가능하게 함
- 점프하면 이 변수는 true가 되고, 땅에 충돌하면 이 변수는 false가 됨

땅과의 충돌 판정

```
private float _speed = 5.0f;
private float _jumpForce = 500.0f;
private Rigidbody2D _rb;
private bool _isJumping = false;

// Update is called once per frame
이벤트 함수
void Update()
{
    transform.Translate( translation: _speed * Time.deltaTime * Input.GetAxis("Horizontal") * Vector3.right);

    if (Input.GetKeyDown(KeyCode.Space) && !_isJumping)
    {
        _isJumping = true;
        _rb.AddForce(Vector2.up * _jumpForce);
    }
}

이벤트 함수
void OnCollisionEnter2D(Collision2D collision)
{
    _isJumping = false;
}
```

땅과의 충돌 판정

- 이 방식의 문제점?
  - 플레이어가 땅의 바닥에 닿았든, 옆면에 닿았든 똑같이 바닥에 닿은 것으로 취급됨 ⇒ 벽타기가 가능해지는 버그
- 실제 플랫포머 게임을 만들 때는 이러한 버그가 발생하지 않도록 적절한 판정 방법을 사용함

이번 주차 요약

- Unity 에디터 다루기
- 씬, 게임오브젝트, 컴포넌트, 에셋
- 스크립트로 오브젝트 움직이기
- 물리엔진 사용법

- 다음 주차: 10/7